MECHATRONICS AND IOT
ASSIGNMENT – 13

Industrial DG Monitoring and Analytics System

PROJECT REPORT

KOWSHIK K 24MER025

NITHISH J 24MER037

KAVIN KRISHNA D S 24MER024

LOGESH S 24MER027

1. Project Overview:

The Cloud-Based Industrial DG Monitoring and Analytics System is designed to provide continuous and intelligent monitoring of diesel generator (DG) performance using IoT, cloud computing, and data analytics. The system enables industrial operators to remotely observe the operating condition of a diesel generator and take timely action when abnormal conditions occur.

The system uses various sensors to measure important generator parameters such as voltage, current, engine temperature, RPM, fuel level, vibration, and operating hours. These sensors are connected to a microcontroller, which continuously collects and processes the sensor readings before transmitting the information to a cloud platform through Wi-Fi or other internet communication technologies.

The cloud platform acts as a centralized location for data storage, visualization, and analysis. A web-based dashboard displays real-time generator parameters using numerical values, graphs, gauges, and status indicators. Historical data is also stored in the cloud, allowing operators to compare generator performance over different periods and identify operating trends.

2. Project Statement:

Industrial diesel generators play a critical role in providing backup and continuous power for factories, hospitals, commercial buildings, data centers, and other mission-critical facilities. The reliable operation of these generators is essential to ensure uninterrupted power supply and maintain industrial productivity. However, traditional DG monitoring methods primarily rely on manual inspections and local control panels, making it difficult for operators to continuously track generator performance and identify potential issues in real time.

The absence of continuous monitoring can result in delayed detection of critical conditions such as overheating, excessive load current, abnormal vibration, low fuel levels, and irregular engine speed. These issues may lead to unexpected equipment failures, increased maintenance costs, reduced operational efficiency, and unplanned downtime. Furthermore, conventional monitoring systems often lack centralized data storage and analytical capabilities, making it challenging to evaluate long-term generator performance and maintenance requirements.

The proposed system is designed to:

- Continuously monitor critical DG operating parameters.

- Provide real-time remote monitoring through a cloud platform.
- Detect abnormal operating conditions automatically.
- Generate alerts for unsafe or critical situations.
- Store historical data for performance analysis.
- Support predictive and preventive maintenance activities.
- Reduce manual inspection requirements.
- Improve operational reliability and equipment safety.
- Enhance overall industrial power management efficiency.

3. Proposed Solution:

The proposed **Cloud-Based Industrial DG Monitoring and Analytics System** provides an intelligent and automated approach for monitoring the operational health and performance of diesel generators. The system integrates multiple sensors, a microcontroller, cloud computing, and data analytics technologies to continuously collect and analyze generator operating parameters in real time.

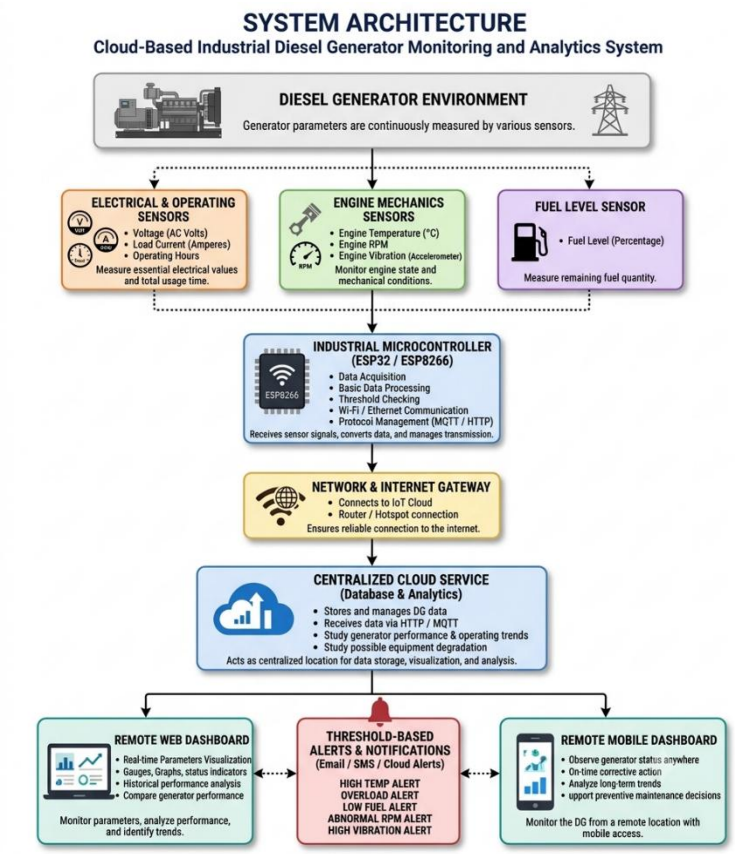
Various sensors are installed on the diesel generator to measure critical parameters such as voltage, current, engine temperature, fuel level, vibration, RPM, and operating hours. These sensors continuously capture the generator's operating conditions and transmit the measured data to an ESP32 or ESP8266 microcontroller. The controller processes the sensor readings, converts them into meaningful values, and prepares the information for transmission to the cloud platform.

The processed data is transmitted through Wi-Fi using communication protocols such as HTTP or MQTT. The cloud platform serves as a centralized repository for storing, managing, and analyzing the collected data. A web-based dashboard provides real-time visualization of generator parameters through graphs, gauges, status indicators, and numerical displays, enabling operators to monitor generator performance remotely from any location.

The proposed solution provides the following features:

- Real-time monitoring of diesel generator parameters.
- Cloud-based data storage and management.
- Remote access through a web-based dashboard.
- Automatic abnormal-condition detection.
- Alert generation for critical operating conditions.

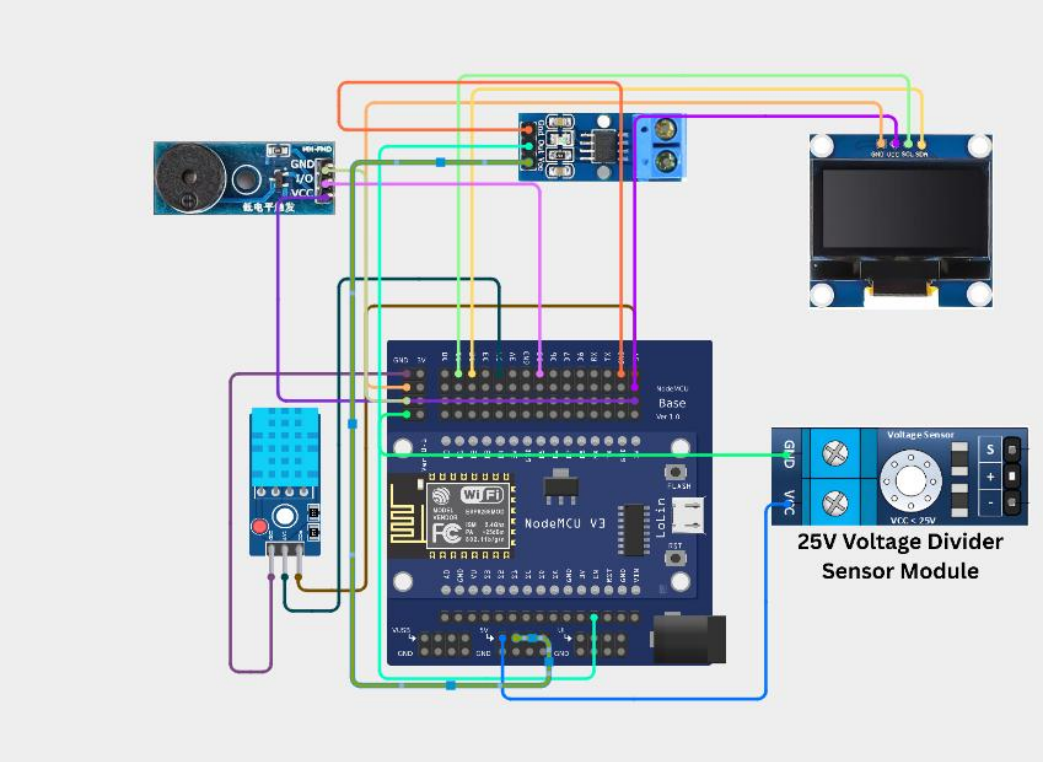
4. System Architecture:



5. Circuit Table:

Component	Connection
DHT11 Sensor	D5
Voltage Sensor	A0
Current Sensor (ACS712)	ADC Input
Fuel Level Sensor	GPIO32
OLED Display	D1, D2
ESP8266/ESP32	Wi-Fi & Processing
Power Supply	3.3V/5V & GND

6. Circuit Design:



7. Algorithm:

Step 1: Start the system and initialize all components.
Step 2: Connect the controller to the Wi-Fi network.
Step 3: Read voltage, current, temperature, fuel level, RPM, and vibration data.
Step 4: Process and validate the sensor readings.
Step 5: Display the measured values on the OLED display.
Step 6: Compare the readings with predefined threshold limits.
Step 7: Detect abnormal operating conditions.
Step 8: Generate alerts when limit violations occur.
Step 9: Upload the sensor data to the cloud platform.
Step 10: Update the cloud dashboard with real-time data.
Step 11: Store the collected data for analysis.
Step 12: Repeat the monitoring process continuously.

8. Coding:

```
#include <WiFi.h> #include <HTTPClient.h> #include <DHT.h>

// Wi-Fi credentials
const char* ssid = "Luky003";
const char* password = "kowshik03";


// ThingSpeak configuration
const char* server = "http://api.thingspeak.com/update"; const char* apiKey =
"Nithish_037";
  #define AIO_KEY "aio_LSVj97Tw74nWq0v3bzxrgwrLg3M1"



// DHT sensor #define DHTPIN 4
#define DHTTYPE DHT11 DHT dht(DHTPIN, DHTTYPE);

// Sensor pins
#define VOLTAGE_PIN 34
#define CURRENT_PIN 35
#define FUEL_PIN 32
#define VIBRATION_PIN 27;


// RPM sensor #define RPM_PIN 26

volatile unsigned long pulseCount = 0; unsigned long previousMillis = 0;

float voltage; float current; float temperature; float humidity; float fuelLevel; float
vibration; float rpm;

void IRAM_ATTR rpmPulse()
{
pulseCount++;
}
```

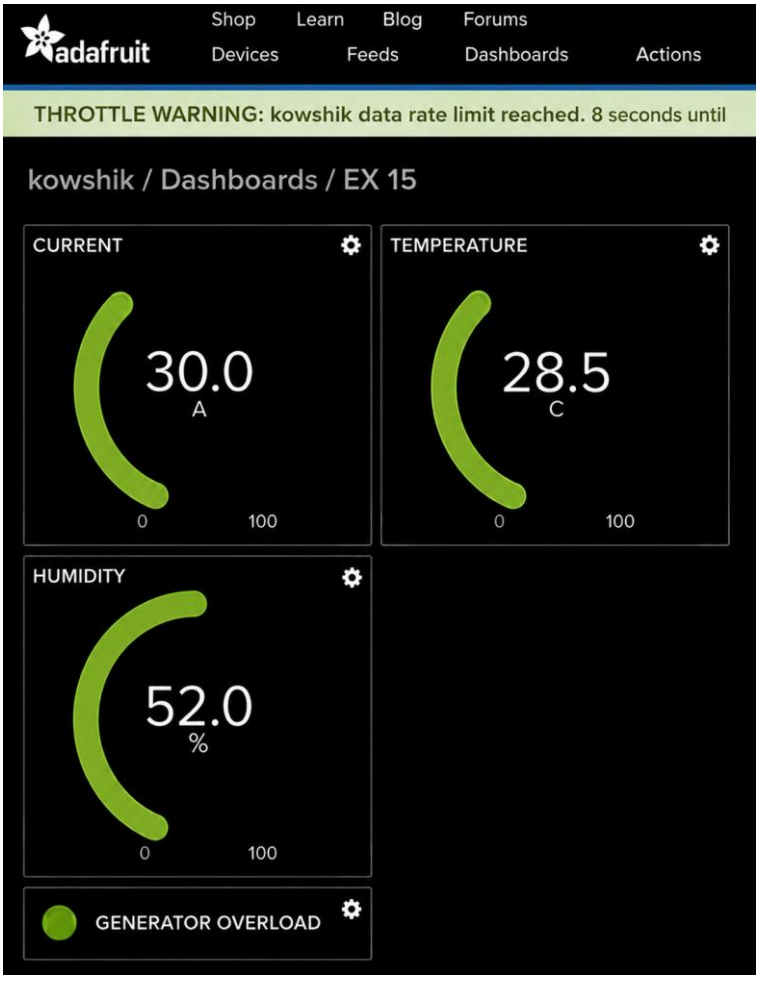
```
void setup()
{
Serial.begin(115200);

dht.begin();


pinMode(VIBRATION_PIN, INPUT); pinMode(RPM_PIN, INPUT_PULLUP);

attachInterrupt(digitalPinToInterrupt(RPM_PIN), rpmPulse, RISING);

WiFi.begin(ssid, password);


Serial.print("Connecting to Wi-Fi");


while (WiFi.status() != WL_CONNECTED)
{
delay(500); Serial.print(".");
}


Serial.println();
Serial.println("Wi-Fi Connected"); Serial.print("IP Address: ");
Serial.println(WiFi.localIP());
}


void loop()
{
// Read temperature and humidity temperature = dht.readTemperature(); humidity =
dht.readHumidity();

// Read analog sensors
int voltageValue = analogRead(VOLTAGE_PIN); int currentValue =
analogRead(CURRENT_PIN); int fuelValue = analogRead(FUEL_PIN);

// Convert sensor values
```

```
voltage = (voltageValue / 4095.0) * 230.0; current = (currentValue / 4095.0) * 50.0;
fuelLevel = (fuelValue / 4095.0) * 100.0;

// Read vibration
vibration = digitalRead(VIBRATION_PIN);


// Calculate RPM pulseCount = 0; delay(1000);
rpm = pulseCount * 60.0;


// Display readings
Serial.println("          ");
Serial.print("Voltage : "); Serial.print(voltage); Serial.println(" V");

Serial.print("Current : "); Serial.print(current); Serial.println(" A");

Serial.print("Temperature : "); Serial.print(temperature);

Serial.println(" C");


Serial.print("Humidity        : "); Serial.print(humidity); Serial.println(" %");

Serial.print("Fuel Level : "); Serial.print(fuelLevel); Serial.println(" %");

Serial.print("RPM     : "); Serial.println(rpm);

Serial.print("Vibration : "); Serial.println(vibration);

// Abnormal condition detection if (temperature > 80)
{
Serial.println("ALERT: High Temperature!");
}


if (current > 45)
{
Serial.println("ALERT: Overload Condition!");
}
```

```
if (fuelLevel < 20)
{
Serial.println("ALERT: Low Fuel Level!");

}


if (rpm < 1000)
{
Serial.println("ALERT: Low RPM!");
}


// Send data to cloud
if (WiFi.status() == WL_CONNECTED)
{
HTTPClient http;


String url = String(server) + "?api_key=" + apiKey + "&field1=" + String(voltage) +
"&field2=" + String(current) +
"&field3=" + String(temperature) + "&field4=" + String(fuelLevel) + "&field5=" +
String(rpm) + "&field6=" + String(vibration);

http.begin(url);


int httpResponseCode = http.GET();


if (httpResponseCode > 0)
{
Serial.print("Cloud Response: "); Serial.println(httpResponseCode);
Serial.println("Data uploaded successfully");

}
else
{
```

```
Serial.println("Cloud upload failed");
}


http.end();
}


delay(15000);
}
```

9. System Working:



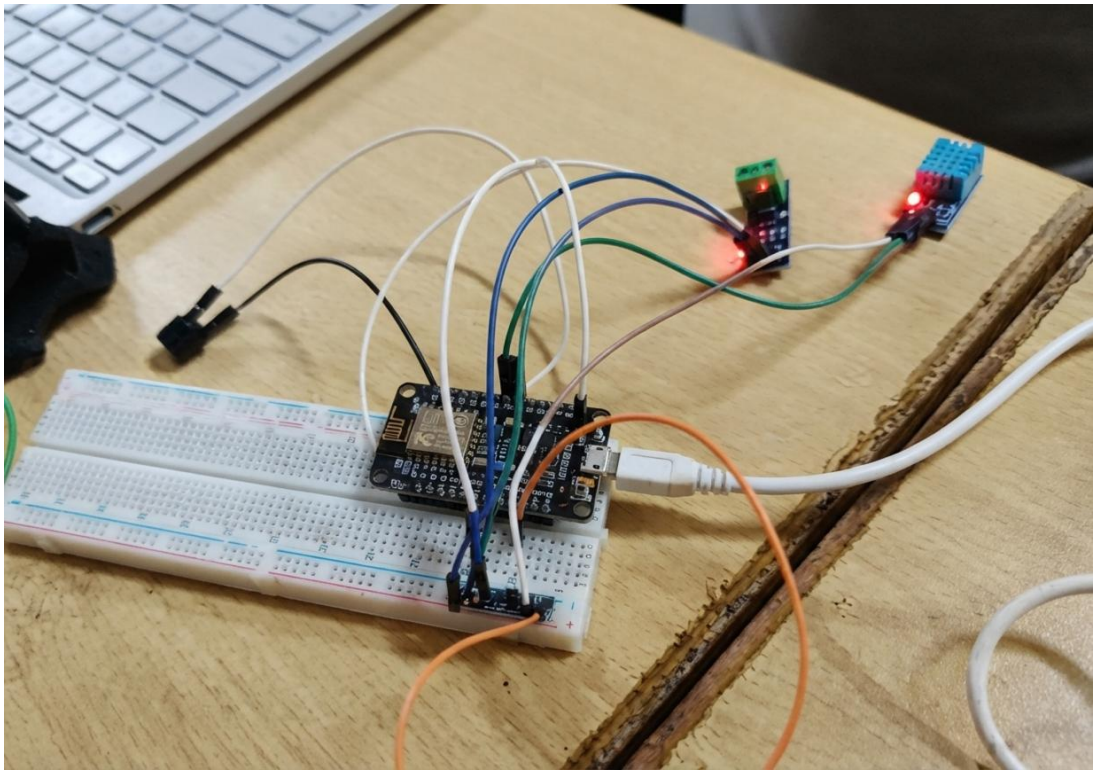
10. Working sequence:

Sensing → Processing → Local Display → MQTT Transmission → Cloud Storage → Data Analysis

11. Bill of Materials:

S.No.	Component	Quantity
1	ESP8266/ESP32 Microcontroller	1
2	DHT11 Temperature Sensor	1
3	Voltage Sensor Module	1
4	ACS712 Current Sensor	1
5	Fuel Level Sensor	1
6	Vibration Sensor	1
7	OLED Display	1
8	Breadboard	1
9	Jumper Wires	1 Set
10	Power Supply	1

12. Prototype Implementation:



13. Results and Performance Analysis:

Performance:

The system provides:

- Real-time data collection and processing.
- Reliable cloud-based data transmission.
- Remote monitoring through a web dashboard.
- Real-time visualization of generator parameters.
- Historical data storage for analysis.
- Automatic abnormal-condition detection.
- Alert generation for critical operating conditions.
- Support for performance analytics and maintenance planning.

Advantages:

- Enables real-time generator monitoring.
- Provides remote access from any location.

- Reduces manual inspection efforts.
- Detects faults at an early stage.
- Generates automatic alerts and notifications.
- Supports preventive and predictive maintenance.
- Stores historical operating data.

Limitations:

- Requires stable internet connectivity.
- Depends on sensor accuracy and reliability.
- Network failures may interrupt data transmission.
- Cloud services may involve additional costs.
- Sensors require periodic calibration.
- Initial setup cost can be relatively high.
- Environmental conditions may affect sensor performance.